

DETECCIÓN DE ANOMALÍAS SIMULADAS EN TRÁFICO AÉREO REAL ADS-B MEDIANTE ELASTIC STACK (SIEM)

https://github.com/mariorfdez97/ADS-B_SIEM

ENTREGA FINAL

ADMINISTRACIÓN DE SISTEMAS



CONTENIDO

1.	Introducción y justificación	4
1.1	Alcance y objetivo de la segunda entrega.....	5
1.2	Estructura del documento.....	6
2.	Fundamentos del Elastic Stack, aplicación y relación con la asignatura.....	6
2.1	De la teoría a la práctica.....	7
3.	Descripción de la arquitectura y módulos del sistema.....	8
3.1	Visión general de la arquitectura	8
3.2	Servicio de adquisición ADS-B.....	9
3.3	Co-ATC (backend y frontend).....	10
3.4	Pipeline de ingesta y procesamiento (Logstash)	10
3.5	Elasticsearch.....	11
3.6	Kibana	11
3.7	Integración SIEM en la interfaz Co-ATC	11
4.	Implementación práctica del SIEM	12
4.1	Despliegue de Elastic Stack	12
4.2	Ingesta y normalización de datos ADS-B	14
4.3	Detección de anomalías de seguridad	16
4.4	Dashboards y Visualización.....	17
4.5	Requisitos Técnicos y de Calidad	19
5.	Gestión del tráfico ADS-B y flujo de datos	20
5.1	Adaptación de Co-ATC al contexto de LEBL	20
5.2	Orquestador de tráfico ADS-B	21

5.3 Implementación del feed ADS-B (servicio adsb-feed)	21
5.4 Escenario operativo y control del ritmo de ingesta.....	25
5.5 Flujo completo y relación con Elastic Stack	25
6. Ejemplo práctico de funcionamiento y operación.....	26
6.1 Escenario de ejemplo	26
6.2 Detección en Kibana (Discover).....	28
7. Relación con la asignatura y resumen de lo realizado	30
8. Conclusiones y pasos siguientes.....	31
9. Referencias y bibliografía	32

1. INTRODUCCIÓN Y JUSTIFICACIÓN

La gestión de eventos e información de seguridad (SIEM) basada en Elastic Stack se ha consolidado como una solución flexible y potente para centralizar logs, correlacionar datos y visualizar alertas en tiempo real, un conjunto de herramientas potente en entornos críticos como la aviación. En la industria aeronáutica se generan volúmenes masivos de datos (planificación de vuelos, telemetría, mantenimiento, etc.), cuyo análisis en tiempo real es clave para mejorar la eficiencia operativa y la seguridad. Elastic Stack – conformado por Elasticsearch, Logstash y Kibana – permite almacenar y consultar grandes conjuntos de datos de forma distribuida, facilitando la detección de patrones anómalos y amenazas mediante búsquedas en tiempo real y o dashboards dinámicos.

Estas características han llevado a su adopción en aviación no solo para análisis operativos, sino también para reforzar la ciberseguridad: por ejemplo, Flightwatching, una empresa aeroespacial francesa, monitorea en esta tecnología datos telemétricos de más de 600 aeronaves en tiempo real, utilizando Kibana para tableros y alertas, e incluso detección de anomalías automatizada sobre dichos datos. Este caso demuestra la capacidad de Elastic Stack para centralizar datos aeronáuticos y descubrir comportamientos fuera de lo normal en tiempo real, mejorando la conciencia situacional y la toma de decisiones en entornos de aviación.

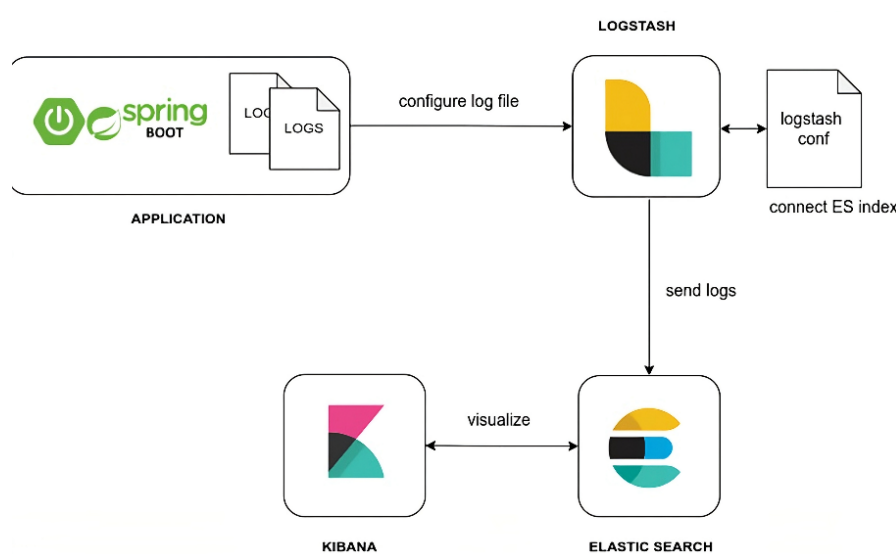


Ilustración 1 - Ejemplo modular de arquitectura SIEM

En este proyecto se aborda un caso concreto: la aplicación de Elastic Stack como SIEM para detectar anomalías en datos de tráfico aéreo ADS-B simulado. ADS-B (Automatic Dependent Surveillance–Broadcast) es una tecnología de vigilancia cooperativa en la cual las aeronaves emiten periódicamente por radio su posición, velocidad, identificador, etc., proporcionando a los controladores información más precisa y en tiempo real que los radares convencionales. Este sistema está adoptado ampliamente a nivel internacional (en España la implantación corre a cargo de ENAIRE).

No obstante, ADS-B carece de mecanismos de seguridad básicos: las transmisiones se emiten en abierto sin autenticación ni cifrado, lo que las hace vulnerables a interferencias maliciosas. Un atacante con equipo radio y conocimiento del protocolo puede inyectar señales falsas en la frecuencia (1090 MHz en aviación comercial) simulando blancos inexistentes o distorsionando datos de aviones reales. Entre las amenazas documentadas se incluyen la suplantación de aeronaves (spoofing) o la creación de aviones fantasma (mensajes inventados que muestran tráfico inexistente), aprovechando la ausencia de autenticación.

Para reforzar la justificación de este proyecto, resulta relevante señalar que en 2017 la FAA emitió alertas dirigidas al personal de mantenimiento, advirtiéndole que las pruebas en tierra de sistemas ADS-B pueden generar “targets fantasma” en los radares de control de tráfico aéreo cuando se transmiten datos de posición sin las precauciones adecuadas. Estos riesgos evidencian la necesidad de herramientas de monitorización que permitan distinguir actividad legítima de anómala en los datos ADS-B, alertando tempranamente sobre posibles incidentes de seguridad aérea.

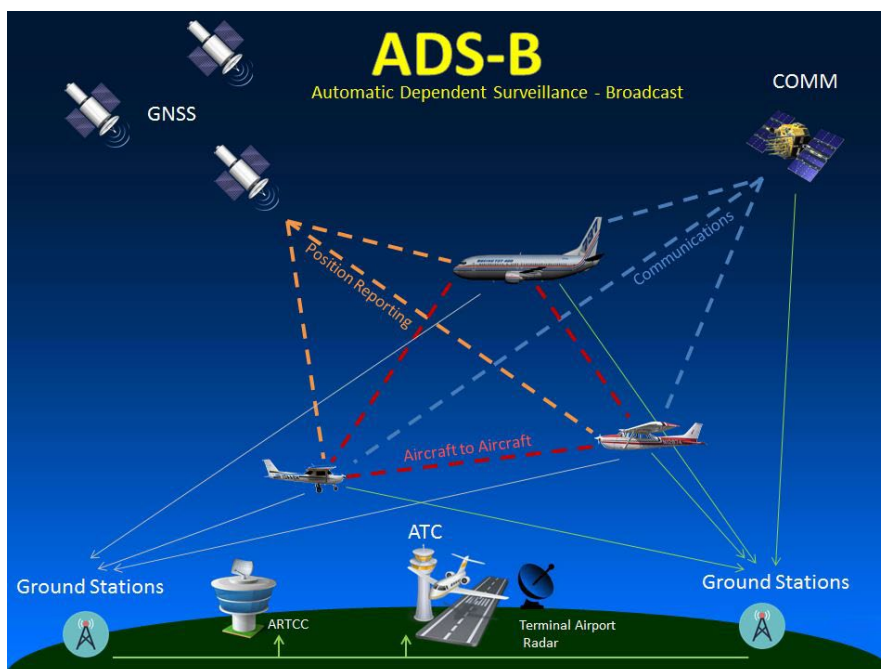


Ilustración 2 - Funcionamiento ADS-B

Un SIEM basado en Elastic Stack resulta idóneo para este cometido: puede correlacionar datos y aplicar reglas de detección para resaltar comportamientos atípicos, complementando la vigilancia tradicional de los controladores.

1.1 ALCANCE Y OBJETIVO DE LA SEGUNDA ENTREGA

Esta segunda entrega traslada la propuesta teórica a una implementación real y reproducible. Se documenta la construcción del laboratorio con Docker, el flujo de datos ADS-B de extremo a extremo (feed → Co-ATC → Logstash → Elasticsearch → Kibana), las reglas SIEM aplicadas y la visualización operativa en dashboards. Además, se describe cómo se notifica al controlador en la interfaz de Co-ATC cuando se detectan anomalías.

El alcance es académico y controlado: se utiliza un laboratorio aislado y datos ADS-B públicos, sin cifrado ni autenticación, con anomalías simuladas para evaluar la detección. No se pretende reemplazar sistemas ATC reales ni realizar certificaciones operativas; el foco está en validar el modelo de ingesta, correlación y visualización, y en mostrar cómo un SIEM puede aportar visibilidad y trazabilidad en un escenario crítico.

1.2 ESTRUCTURA DEL DOCUMENTO

El documento se organiza de forma incremental: primero se describe el despliegue y los componentes del stack, después el proceso de ingesta y normalización, las reglas SIEM y los dashboards; finalmente se presenta un ejemplo práctico de detección, la integración de alertas en Co-ATC, las complicaciones encontradas y la relación con el temario.

En resumen, el proyecto busca demostrar cómo la utilización de Elastic Stack como SIEM puede fortalecer la seguridad en entornos de aviación civil, aportando visibilidad unificada sobre datos críticos y capacidad de respuesta ante anomalías, todo ello mediante una implementación reproducible en un laboratorio académico.

2. FUNDAMENTOS DEL ELASTIC STACK, APLICACIÓN Y RELACIÓN CON LA ASIGNATURA

El Elastic Stack SIEM es un entorno avanzado de gestión de información y eventos de seguridad (SIEM) que integra diversos principios estudiados en la asignatura, combinando conocimientos de administración de sistemas, monitorización de redes, gestión de logs, virtualización y seguridad de la información. Su implementación práctica en este proyecto no solo sirve como ejercicio técnico, sino también como aplicación directa de los fundamentos teóricos abordados en clase.

El Elastic Stack — compuesto principalmente por Elasticsearch, Logstash y Kibana — traduce estos conceptos teóricos en una arquitectura modular y escalable para el tratamiento de datos. A continuación, se explican mas en detalle los módulos dichos anteriormente.

ELASTICSEARCH

Elasticsearch funciona como el repositorio central donde se almacenan los mensajes ADS-B simulados. Su estructura de índices distribuidos permite buscar y analizar los datos en tiempo real, aplicando los principios de almacenamiento eficiente y gestión de registros de eventos estudiados en la asignatura.

LOGSTASH

Se encarga del procesamiento intermedio y aplica conceptos de automatización de tareas y transformación de flujos de datos, vistos en los temas de automatización de sistemas. Gracias a sus filtros y pipelines, muestra cómo crear un flujo controlado que convierte datos brutos en información estructurada y útil.

ELASTIC SECURITY (KIBANA)

Integra los temas de visualización, análisis y gestión de incidentes, actuando como una interfaz que ayuda al administrador a interpretar datos y tomar decisiones con fundamento. Su módulo Elastic Security aplica directamente conceptos como detección de anomalías, correlación de eventos y respuesta ante incidentes, tratados en la teoría sobre seguridad de la información y auditoría de sistemas.

Desde la visión del temario, el Elastic Stack SIEM también se apoya en los fundamentos de la virtualización y contenedorización de servicios. El proyecto utilizará Docker para desplegar de forma aislada cada componente del stack, lo cual refuerza el aprendizaje práctico sobre gestión de entornos virtualizados, configuración mediante contenedores y automatización de despliegues, permitiendo una administración más eficiente y reproducible del sistema.

En términos de seguridad, la implementación del SIEM ejemplifica los conceptos teóricos sobre detección de intrusiones y gestión de incidentes. Los mecanismos de correlación de esta tecnología permiten aplicar las técnicas de detección basada en reglas y detección de anomalías estudiadas en el curso. En el contexto del proyecto, estas reglas se traducen en la identificación de comportamientos anómalos en los datos ADS-B, como altitudes negativas o duplicaciones de identificadores ICAO.

Por otra parte, el uso de logs estructurados y análisis de datos en tiempo real conecta directamente con los temas del temario relacionados con administración de registros del sistema (syslog, journald) y gestión de la información en entornos distribuidos. En el proyecto, la simulación de tráfico aéreo genera flujos de datos continuos que son tratados como si provinieran de un entorno real de red o de infraestructura crítica.

2.1 DE LA TEORÍA A LA PRÁCTICA

En la primera entrega se planteó Elastic Stack como una solución SIEM aplicable a ADS-B. En esta implementación, el enfoque se materializa en un laboratorio reproducible: los datos ADS-B se ingieren continuamente, se normalizan, se etiquetan con reglas de seguridad y se visualizan en Kibana. La correlación no se limita a valores aislados (p. ej. velocidad alta), sino que incorpora consistencia física y contextual (RSSI vs distancia, altitud negativa, squawk de emergencia), lo que convierte el caso en un ejemplo realista de análisis de telemetría aeronáutica.

El Elastic Stack SIEM actúa como un puente entre la teoría y la práctica de la asignatura. Integra en un entorno funcional todos los elementos que se abordan a lo largo del curso:

1. Principios de administración de sistemas y servicios.
2. Virtualización y despliegue mediante contenedores.
3. Monitorización y análisis de logs.
4. Detección y gestión de incidentes de seguridad.
5. Automatización del control y respuesta ante eventos.

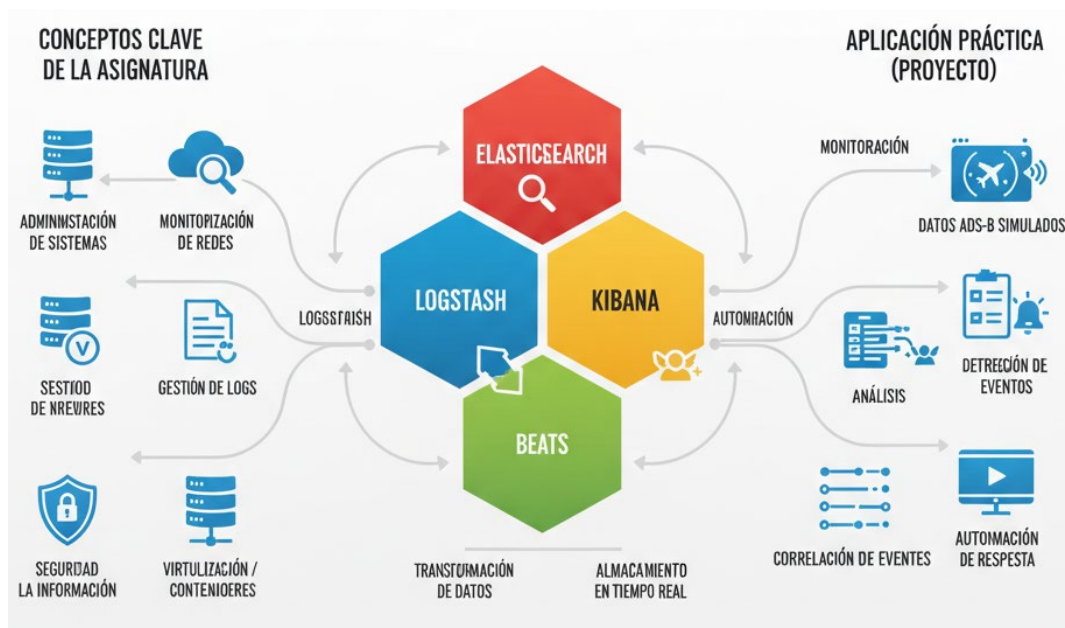


Ilustración 3 - Relación entre Elastic Stack y la asignatura

3. DESCRIPCIÓN DE LA ARQUITECTURA Y MÓDULOS DEL SISTEMA

El sistema desarrollado se basa en una **arquitectura modular y distribuida**, diseñada para integrar la adquisición de datos ADS-B reales, su visualización operativa y su análisis mediante un pipeline SIEM basado en Elastic Stack. Cada módulo cumple una función bien definida dentro del flujo global, facilitando la mantenibilidad, la trazabilidad de los datos y la posibilidad de ampliación futura.

3.1 VISIÓN GENERAL DE LA ARQUITECTURA

La arquitectura del laboratorio se organiza en torno a cinco bloques principales:

- Servicio de adquisición ADS-B
- Backend y frontend Co-ATC
- Pipeline de ingesta y procesamiento (Logstash)
- Almacenamiento y análisis (Elasticsearch)
- Visualización y explotación (Kibana e interfaz de alertas en Co-ATC)

Todos los componentes se despliegan mediante contenedores Docker y se comunican a través de una red interna, garantizando un entorno reproducible y aislado.

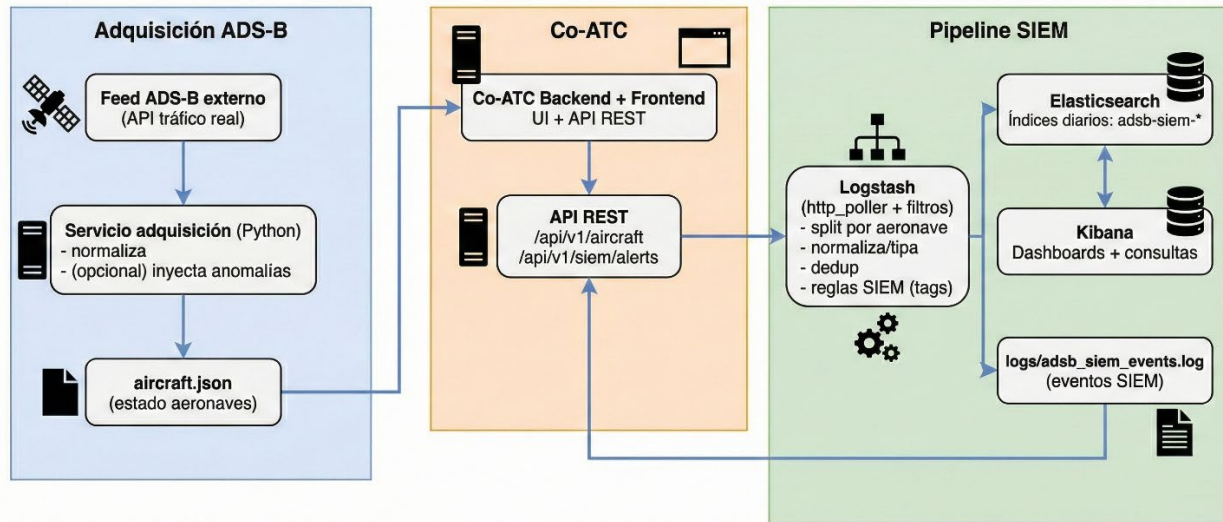


Ilustración 4 - Descripción modular general SIEM

3.2 SERVICIO DE ADQUISICIÓN ADS-B

El servicio de adquisición ADS-B es el encargado de obtener datos de tráfico aéreo real desde una fuente externa y actuar como punto de entrada del sistema. Está implementado en Python y consulta periódicamente la API ADS-B externa con un intervalo configurable, lo que permite controlar el ritmo de ingesta y garantizar la estabilidad del laboratorio.

Los datos recibidos se filtran y normalizan, seleccionando únicamente los campos relevantes y adaptándolos al formato utilizado por Co-ATC. De forma opcional, el servicio permite la inyección controlada de anomalías, registrando dichas alteraciones para su posterior trazabilidad.

Finalmente, el estado consolidado de las aeronaves se almacena en un fichero `aircraft.json`, que es usado directamente por Co-ATC.

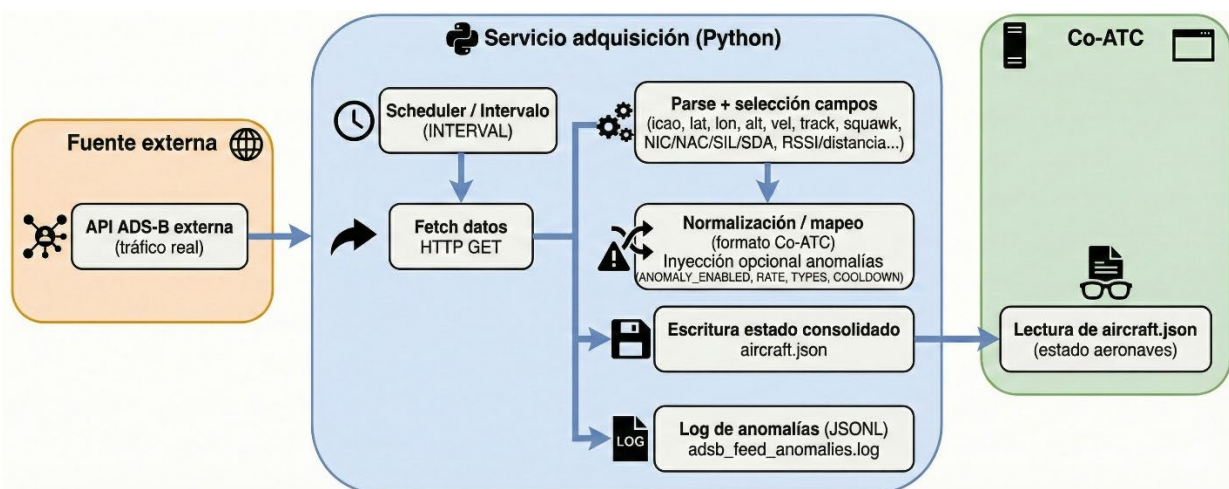


Ilustración 5 - Arquitectura interna y flujo del servicio de adquisición ADS-B

3.3 CO-ATC (BACKEND Y FRONTEND)

Co-ATC actúa como núcleo el fichero aircraft.json, mantiene el estado interno de las aeronaves y ofrece una visualización en tiempo casi real mediante una interfaz web. Además, expone una API REST que permite a otros módulos acceder al estado de los vuelos y a las alertas SIEM.

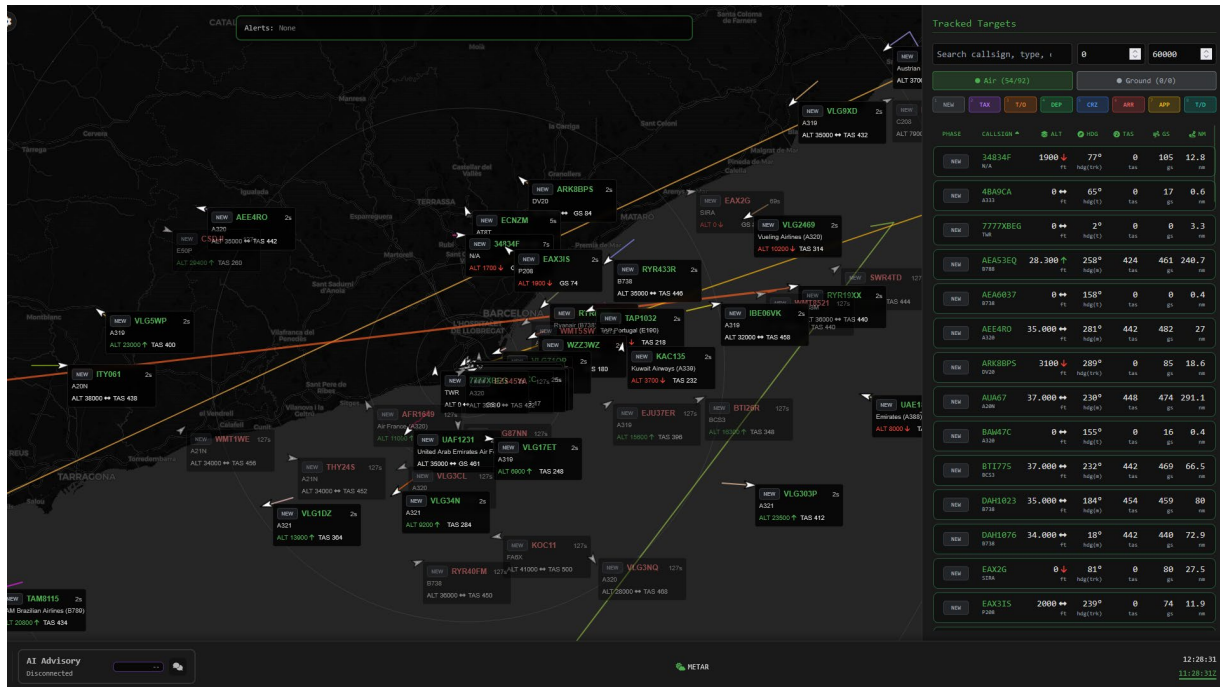


Ilustración 6 – Panel de control de tráfico aéreo (Co-ATC) y monitorización de aeronaves

3.4 PIPELINE DE INGESTA Y PROCESAMIENTO (LOGSTASH)

Logstash actúa como el **núcleo del sistema SIEM**, siendo responsable de la ingesta, procesamiento y enriquecimiento de los datos ADS-B expuestos por Co-ATC. Este componente consulta periódicamente la API REST interna de Co-ATC y transforma la información recibida en eventos individuales por aeronave.

Durante el procesamiento, Logstash normaliza y tipa los campos ADS-B, calcula métricas derivadas y elimina estados redundantes mediante mecanismos de deduplicación. Además, aplica reglas de detección de anomalías basadas en umbrales y relaciones físicas entre parámetros de vuelo, etiquetando los eventos anómalos con marcas SIEM.

Los eventos resultantes se envían a Elasticsearch para su almacenamiento y análisis, y de forma complementaria se registran en logs estructurados locales, que son utilizados para la visualización de alertas en la interfaz de Co-ATC. Este enfoque permite una detección temprana de anomalías y una integración directa con los sistemas de visualización.

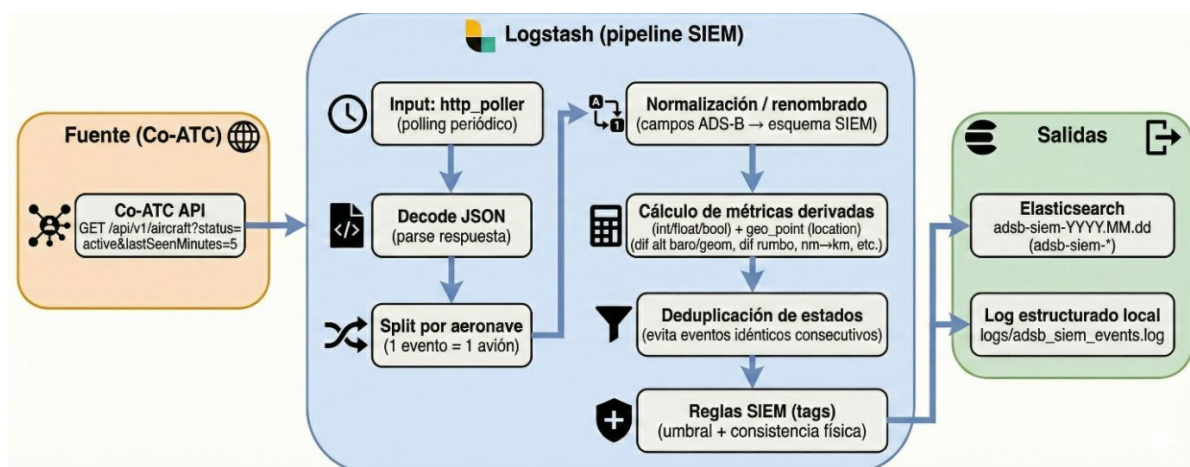


Ilustración 7 - Pipeline de Logstash para el procesamiento y enriquecimiento de eventos

3.5 ELASTICSEARCH

Elasticsearch proporciona el almacenamiento persistente y el motor de búsqueda y agregación. Los eventos ADS-B se indexan en índices diarios, permitiendo consultas eficientes, análisis temporal y geoespacial, y la construcción de visualizaciones avanzadas.

3.6 KIBANA

Kibana se utiliza como herramienta principal de visualización y análisis. A partir de los índices generados en Elasticsearch, se construyen dashboards, gráficos y consultas que permiten monitorizar el tráfico aéreo y detectar patrones anómalos de forma visual e intuitiva.

3.7 INTEGRACIÓN SIEM EN LA INTERFAZ CO-ATC

Como complemento a Kibana, el sistema incorpora un módulo de alertas integrado directamente en la interfaz de Co-ATC. Este módulo consulta eventos SIEM almacenados en logs estructurados y presenta alertas en tiempo casi real al usuario, cerrando el ciclo de detección y visualización operativa.

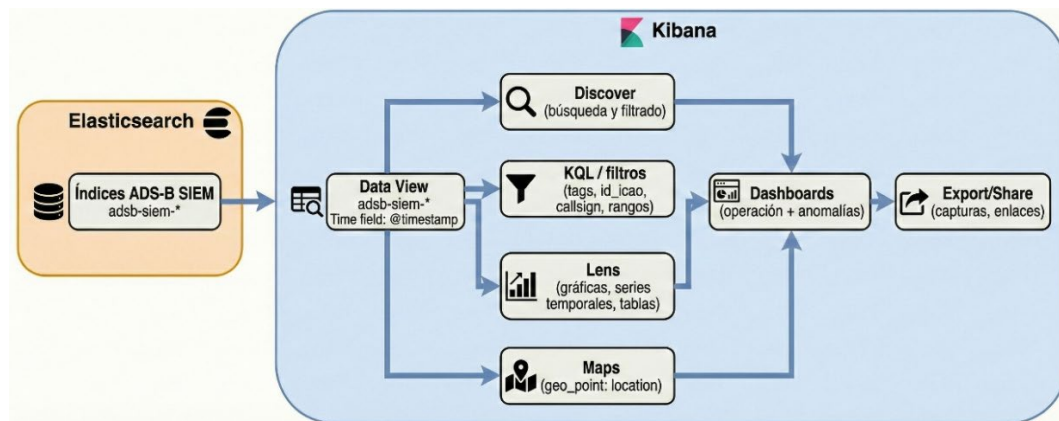


Ilustración 8 - Flujo de visualización y explotación de datos en Kibana

4. IMPLEMENTACIÓN PRÁCTICA DEL SIEM

En esta segunda entrega, el objetivo principal es trasladar la propuesta teórica a una implementación real del SIEM con Elastic Stack. Esto implica definir requisitos claros, demostrar cómo se cumplen en el laboratorio y dejar trazabilidad entre el diseño y la ejecución. El apartado se centra en lo estrictamente técnico: ingestión de datos ADS-B, transformación y correlación en Logstash, indexación en Elasticsearch, visualización en Kibana y notificación en la interfaz Co-ATC.

Para ordenar la implementación, se definen requisitos funcionales y no funcionales. Los funcionales garantizan que el sistema opere de extremo a extremo; los no funcionales aseguran estabilidad, rendimiento y reproducibilidad.

1. Requisitos funcionales

- ✓ R1: ingestión periódica de tráfico ADS-B en formato estandarizado.
- ✓ R2: inyección controlada de anomalías para simular ataques o condiciones anómalas.
- ✓ R3: correlación SIEM y etiquetado automático de eventos.
- ✓ R4: visualización y análisis en Kibana mediante dashboards.
- ✓ R5: notificación de alertas en la interfaz de Co-ATC.

2. Requisitos no funcionales

- RN1: reproducibilidad mediante contenedores Docker.
- RN2: control del ritmo de ingesta y límites de frecuencia.
- RN3: trazabilidad de eventos mediante logs estructurados.
- RN4: separación modular de servicios para facilitar el mantenimiento.

4.1 DESPLIEGUE DE ELASTIC STACK

El laboratorio se despliega mediante Docker Compose, lo que permite aislar cada componente y reproducir el entorno completo en cualquier máquina. Esta aproximación refuerza la modularidad del sistema y facilita tanto el arranque como la depuración. El archivo `docker-compose.yml` define los servicios principales del stack (Elasticsearch, Logstash y Kibana), junto con los servicios específicos del proyecto (`adsb-feed` y `Co-ATC`).

El despliegue se basa en una arquitectura por contenedores conectados en la misma red interna de Docker. Cada servicio expone puertos específicos al host para permitir el acceso local (Kibana en 5601, Elasticsearch en 9200, Co-ATC en 8000, etc.), y utiliza volúmenes para persistir datos o compartir ficheros entre servicios.

```

1 version: "3.8"
2
3 services:
4   adsb-feed:
5     build:
6       context: ./services/adsb-feed
7     restart: unless-stopped
8     environment:
9       - ANOMALY_ENABLED=true
10      - ANOMALY_RATE=0.01
11      - ANOMALY_MAX_PER_WRITE=1
12      - ANOMALY_COOLDOWN_CYCLES=30
13      - ANOMALY_RSSI_DISTANCE_RATE=0.01
14      - ANOMALY_RSSI_DISTANCE_MAX_PER_WRITE=1
15      - ANOMALY_TYPES=altitud_negativa,velocidad_excesiva,salto_posicion,squawk_emergencia,posible_spoofing
16      - ANOMALY_LOG=/logs/adsb_feed_anomalies.log
17      - INTERVAL=3
18     volumes:
19       - aircraft_data:/data
20       - ./logs:/logs
21     ports:
22       - "9000:9000"
23
24   co-atc:
25     build:
26       context: ./co-atc-main
27     restart: unless-stopped
28     depends_on:
29       - adsb-feed
30     environment:
31       - COATC_CONFIG=/app/configs/config.docker.toml
32     volumes:
33       - ./co-atc-main/configs/config.docker.toml:/app/configs/config.docker.toml:ro
34       - coatc_data:/app/data
35       - ./logs:/logs:ro
36     ports:
37       - "8000:8000"
38
39   elasticsearch:
40     image: docker.elastic.co/elasticsearch/elasticsearch:8.14.1
41     restart: unless-stopped
42     environment:
43       - discovery.type=single-node
44       - xpack.security.enabled=false
45       - ES_JAVA_OPTS=-Xms1g -Xmx1g
46     ulimits:
47       memlock:
48         soft: -1
49         hard: -1
50       nofile:
51         soft: 65536
52         hard: 65536
53     ports:
54       - "9200:9200"
55     volumes:
56       - esdata:/usr/share/elasticsearch/data
57     healthcheck:
58       test: ["CMD-SHELL", "curl -fs http://localhost:9200/_cluster/health || exit 1"]
59       interval: 10s
60       timeout: 5s
61       retries: 10
62
63   kibana:
64     image: docker.elastic.co/kibana/kibana:8.14.1
65     restart: unless-stopped
66     depends_on:
67       elasticsearch:
68         condition: service_healthy
69     environment:
70       - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
71       - XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY=0123456789abcdef0123456789abcdef
72       - XPACK_SECURITY_ENCRYPTIONKEY=0123456789abcdef0123456789abcdef
73       - XPACK_REPORTING_ENCRYPTIONKEY=0123456789abcdef0123456789abcdef
74     ports:
75       - "5601:5601"
76
77   logstash:
78     image: docker.elastic.co/logstash/logstash:8.14.1
79     restart: unless-stopped
80     depends_on:
81       elasticsearch:
82         condition: service_healthy
83       co-atc:
84         condition: service_started
85     user: root
86     command: ["-w", "1"]
87     environment:
88       - LS_JAVA_OPTS=-Xms512m -Xmx512m
89     volumes:
90       - ./infra/logstash/pipeline:/usr/share/logstash/pipeline:ro
91       - ./logs:/logs
92     ports:
93       - "5044:5044"
94       - "9600:9600"
95     healthcheck:
96       test: ["CMD-SHELL", "curl -fs http://localhost:9600/_node/pipelines || exit 1"]
97       interval: 10s
98       timeout: 5s
99       retries: 10
100
101 volumes:
102   esdata:
103   coatc_data:
104   aircraft_data:
105

```

4.2 INGESTA Y NORMALIZACIÓN DE DATOS ADS-B

Una vez desplegada la infraestructura base, el objetivo se centra en la implementación de una ingesta real, continua y normalizada de datos ADS-B, basada en tráfico aéreo real y no en eventos simulados mediante archivos de log estáticos, ***tal y como se planteó inicialmente en la primera entrega.***

En la implementación final, la fuente principal de datos es un **feed ADS-B real** (*explicado en el punto 4 de este informe*), obtenido mediante un servicio auxiliar que consulta periódicamente una API externa (<https://opendata.adsb.fi/api/v3>) de tráfico aéreo y genera un fichero *aircraft.json* con el estado actualizado de las aeronaves. Este fichero es consumido directamente por el backend Co-ATC, que actúa como punto central de agregación y exposición de la información ADS-B a través de una API HTTP REST.

De este modo, la ingesta SIEM se apoya en el estado operativo real mostrado al usuario en Co-ATC, garantizando coherencia entre la visualización y los datos analizados.

La canalización de datos en la implementación final se gestiona de la siguiente manera:

3. Co-ATC como fuente de ingesta lógica:

El backend de Co-ATC actúa como agregador central del tráfico ADS-B. Su endpoint GET `/api/v1/aircraft` devuelve periódicamente la lista de aeronaves activas con todos los campos necesarios para el análisis (posición, altitud, velocidad, señal, integridad, etc.). De este modo, Co-ATC sustituye al simulador propuesto en la primera entrega y se convierte en la fuente única de verdad del sistema.

4. Logstash como motor de ingesta y procesamiento:

Logstash (`/infra/logstash.conf`) utiliza un input `http_poller` para consultar periódicamente la API de Co-ATC. La respuesta JSON se divide en eventos individuales (uno por aeronave), se normaliza y se enriquece con campos derivados. Este enfoque elimina la necesidad de Filebeat (planteado inicialmente) y simplifica la arquitectura al trabajar con una fuente estructurada en tiempo real.

```

1 input {
2   http_poller {
3     urls => {
4       "co_atc_api" => {
5         method => "get"
6         url => "http://co-atc:8000/api/v1/aircraft?status=active&lastSeenMinutes=5"
7         headers => {
8           "Accept" => "application/json"
9         }
10      }
11    }
12    request_timeout => 10
13    schedule => { every => "5s" }
14    codec => "json"
15    ecs_compatibility => "v8"
16    tags => ["co-atc-api"]
17  }
18 }

```



```

1 # 3. RENOMBRAMIENTO MASIVO (Nuevos campos añadidos)
2 mutate {
3   rename => {
4     "[aircraft][hex]" => "id_icao"
5     "[aircraft][flight]" => "callsign_vuelo"
6     "[aircraft][adsb][r]" => "matricula_aeronave"
7     "[aircraft][adsb][t]" => "modelo_aeronave"
8     "[aircraft][airline]" => "aerolinea"
9
10    "[aircraft][status]" => "estado"
11    "[aircraft][on_ground]" => "en_tierra"
12    "[aircraft][phase][current][0][phase]" => "fase_vuelo"
13    "[aircraft][date_tookoff]" => "hora_despegue"
14    "[aircraft][date_landed]" => "hora_aterrizaje"
15
16    "[aircraft][adsb][lat]" => "latitud"
17    "[aircraft][adsb][lon]" => "longitud"
18    "[aircraft][adsb][track]" => "rumbo_real"
19    "[aircraft][adsb][mag_heading]" => "rumbo_magnetico"
20    "[aircraft][distance]" => "distancia_receptor_nm"
21
22    "[aircraft][adsb][alt_baro]" => "altitud_pies"
23    "[aircraft][adsb][alt_geom]" => "altitud_geometrica_pies"
24    "[aircraft][adsb][baro_rate]" => "tasa_vertical_pies_min"
25    "[aircraft][adsb][nav_altitude_mcp]" => "altitud_objetivo_piloto"
26
27    "[aircraft][adsb][gs]" => "velocidad_tierra_nudos"
28    "[aircraft][adsb][ias]" => "velocidad_indicada"
29    "[aircraft][adsb][tas]" => "velocidad_verdadera"
30    "[aircraft][adsb][mach]" => "numero_mach"
31    "[aircraft][adsb][roll]" => "angulo_alabeo"
32
33    "[aircraft][adsb][rssi]" => "intensidad_senal_rssi"
34    "[aircraft][adsb][nic]" => "integridad_navegacion_nic"
35    "[aircraft][adsb][nac_p]" => "precision_posicion_nacp"
36    "[aircraft][adsb][squawk]" => "codigo_transpondedor"
37    "[aircraft][adsb][category]" => "categoria_estela"
38  }
39 }

```

5. Elasticsearch como almacenamiento y motor analítico:

Los eventos procesados se indexan en Elasticsearch bajo índices diarios (adsb-siem-YYYY.MM.dd). Los campos se tipan correctamente (numéricos, temporales y geoespaciales), permitiendo búsquedas eficientes, agregaciones temporales y visualización geográfica en Kibana.

Con este flujo, el sistema opera en tiempo casi real, con una latencia mínima entre la actualización del estado en Co-ATC y su disponibilidad en Elasticsearch, dashboards de Kibana y alertas SIEM.

4.3 DETECCIÓN DE ANOMALÍAS DE SEGURIDAD

Una vez los datos ADS-B son ingeridos, normalizados e indexados, la detección de anomalías de seguridad en la implementación final se realiza **durante la fase de procesamiento en Logstash**, en lugar de apoyarse en el módulo Elastic Security de Kibana, tal y como se propuso inicialmente en la primera entrega.

Este cambio de enfoque responde a una decisión de diseño orientada a **simplificar el pipeline**, mantener la detección estrechamente ligada a la ingesta de datos y facilitar la integración directa de alertas en la interfaz operativa de Co-ATC. De este modo, la detección de anomalías se aproxima a un modelo de SIEM basado en reglas en tiempo de ingestión, ampliamente utilizado en entornos de monitorización y análisis de logs.

El objetivo de este subsistema es identificar, clasificar y etiquetar comportamientos anómalos o potencialmente maliciosos en las transmisiones ADS-B, que puedan indicar errores en los datos recibidos, inconsistencias físicas o posibles intentos de spoofing.

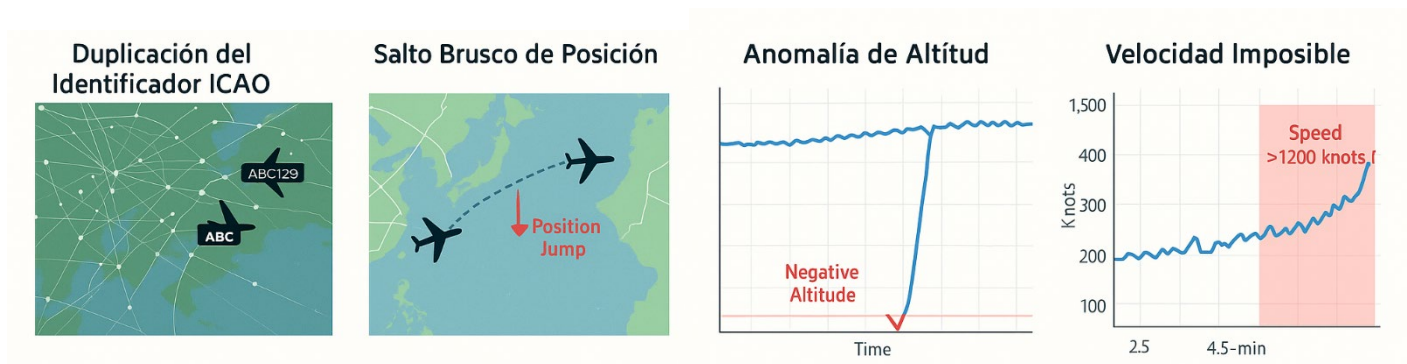


Ilustración 9 - Ejemplos visuales de posibles anomalías

- **Altitudes negativas o fuera de rango operativo**, que pueden indicar errores de sensor, datos corruptos o estados incoherentes.
- **Velocidades imposibles**, superiores a umbrales físicamente razonables para aeronaves convencionales.
- **Incoherencias entre altitud barométrica y geométrica**, que sugieren errores de referencia o degradación de la calidad de navegación.
- **Tasas verticales extremas o inestables**, incompatibles con maniobras normales de vuelo.
- **Incoherencias entre intensidad de señal (RSSI) y distancia al receptor**, patrón comúnmente asociado a datos falsificados o recepción anómala.
- **Códigos squawk de emergencia (7500, 7600, 7700)**, tratados como alertas críticas de seguridad operacional.

Cada una de estas anomalías se implementa como una **regla de detección independiente dentro de la pipeline de Logstash**, mediante filtros condicionales que evalúan los campos del evento ADS-B una vez normalizados. Cuando se cumple una condición de detección, el evento es etiquetado con una o varias **banderas SIEM (tags)** que identifican el tipo de anomalía detectada.


```

1      dist_nm = event.get('distancia_receptor_nm')
2      if dist_nm
3          event.set('distancia_receptor_km', dist_nm.to_f * 1.852)
4      end
5
6      v_tierra = event.get('velocidad_tierra_nudos')
7      if v_tierra && v_tierra.to_f > 1200
8          event.tag('alerta_velocidad_excesiva')
9      end
10
11     alt = event.get('altitud_pies')
12     ias = event.get('velocidad_indicada')
13     if alt && ias && alt.to_f < 10000 && ias.to_f > 260
14         event.tag('alerta_velocidad_baja_cota')
15     end
16
17     if alt && alt.to_f < -100
18         en_tierra = event.get('en_tierra')
19         if en_tierra == false || en_tierra.nil?
20             event.tag('alerta_altitud_negativa')
21         end
22     end

```

4.4 DASHBOARDS Y VISUALIZACIÓN

Una parte esencial del proyecto es la creación de dashboards interactivos en Kibana, que servirán como interfaz visual para la monitorización del tráfico aéreo y la gestión de alertas. Estos paneles permitirán al analista observar tanto la actividad operativa del espacio aéreo simulado como las incidencias de seguridad detectadas por el SIEM.

Los dashboards principales incluirán:

Creación de Data View y campo temporal

Antes de construir visualizaciones, es imprescindible crear un data view que apunte a los índices generados por el SIEM. En Kibana se realiza desde Stack Management → Data Views → Create data view. En este proyecto se utiliza el patrón `adsb-siem-*`, ya que Logstash indexa en Elasticsearch con formato diario (`adsb-siem-YYYY.MM.dd`).

El campo temporal recomendado es `@timestamp`, que Logstash genera a partir de `aircraft.last_seen`, lo que permite que las búsquedas y gráficos se ordenen por el instante real de detección. Una vez creado el data view, es importante ejecutar Refresh field list para que Kibana detecte todos los campos y tipos.

Como verificación rápida, se recomienda comprobar que los campos críticos aparecen con el tipo adecuado:

- `altitud_pies`, `velocidad_tierra_nudos`, `intensidad_senal_rssi` como numéricos.
- `latitud` y `longitud` como numéricos.
- `tags` como array de texto.

Además, es recomendable fijar un rango temporal inicial (por ejemplo, “Últimos 15 minutos”) y guardar una búsqueda base para reutilizarla en varios paneles.

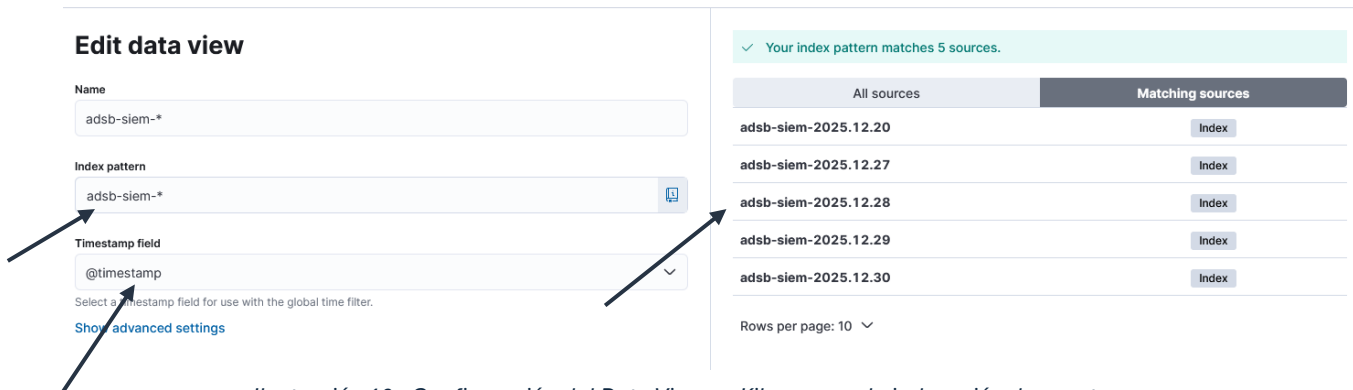


Ilustración 10 - Configuración del Data View en Kibana para la indexación de eventos

Consultas y filtros KQL recomendados

KQL permite filtrar eventos en Discover y en dashboards de forma rápida y legible. En este proyecto se utiliza para aislar tipos de anomalías, aeronaves concretas o rangos físicos (altitud, velocidad, RSSI).

Es importante recordar que KQL distingue tipos numéricos, texto y arrays.

Ejemplos básicos:

- Buscar todas las alertas:
 - tags: "alerta_*
- Solo alertas de velocidad excesiva:
 - tags: "alerta_velocidad_excesiva"
- Filtrar una aeronave concreta por ICAO:
 - id_icao: "4BCE15"
- Filtrar por callsign parcial:
 - callsign_vuelo: "IBE*"

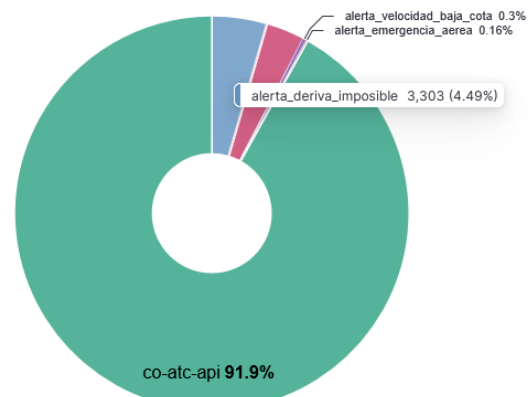


Ilustración 11 - Distribución porcentual de alertas SIEM y eventos de la API Co-ATC

Ejemplos numéricos y combinados:

- Altitud físicamente imposible:
 - altitud_pies < -100
- Velocidad extrema:
 - velocidad_tierra_nudos > 1200
- Spoofing por RSSI y distancia incoherente:
 - intensidad_senal_rssi >= -5 and distancia_receptor_nm >= 200
- Baja integridad de navegación:
 - integridad_navegacion_nic < 6

Operadores útiles:

- and, or, not para combinar condiciones.
- Paréntesis para agrupar:
 - tags: ("alerta_velocidad_excesiva" or "alerta_altitud_negativa")

- Existencia de campo:
 - rssi:* o tags:*

En dashboards, estos filtros se pueden aplicar como filter pills para cada panel, o guardar una búsqueda en Discover y reutilizarla como fuente de datos. Esto permite definir paneles especializados (p. ej., “solo emergencias” o “solo spoofing”), y mantener consistencia entre visualizaciones.

Edit rule

Index threshold

Alert when an aggregated query meets the threshold. [Learn more](#)

Select indices

INDEX adsb-siem-*

Define the condition

WHEN count()

OVER all documents

IS ABOVE 0

FOR THE LAST 5 minutes

Filter

tags : "alerta-velocidad_excesiva"

Optional

Use a KQL expression to limit the scope of your alerts.

Cancel

Show API request

Save

Ilustración 12 - Configuración de una regla de detección basada en umbrales (Index threshold) en Kibana

4.5 REQUISITOS TÉCNICOS Y DE CALIDAD

El sistema deberá cumplir los siguientes requisitos globales para asegurar su correcto funcionamiento:

- **Integridad de datos:** la ingesta debe ser determinista y consistente. Logstash debe procesar cada evento una sola vez y conservar la coherencia entre el estado mostrado en Co-ATC y el estado indexado en Elasticsearch.
- **Fiabilidad:** los contenedores deben poder reiniciarse de forma independiente sin perder datos persistentes (volúmenes), y Logstash debe reintentar envíos cuando Elasticsearch no esté disponible temporalmente.

- **Usabilidad:** Kibana debe proporcionar dashboards claros con mapas y series temporales, permitiendo filtrar por ICAO, rango temporal y tipo de alerta.
- **Portabilidad:** todo el entorno debe ser reproducible con un único docker-compose up, garantizando consistencia entre desarrollo y laboratorio.

5. GESTIÓN DEL TRÁFICO ADS-B Y FLUJO DE DATOS

Para construir un entorno de ingesta de tráfico aéreo que resultara didáctico y técnicamente manejable, se decidió partir del proyecto open-source Co-ATC, el cual ya funciona como backend de monitorización aeronáutica. De este modo, se reduce el tiempo invertido en aspectos ajenos a los objetivos principales del proyecto y focalizando el trabajo en la administración y explotación del SIEM.

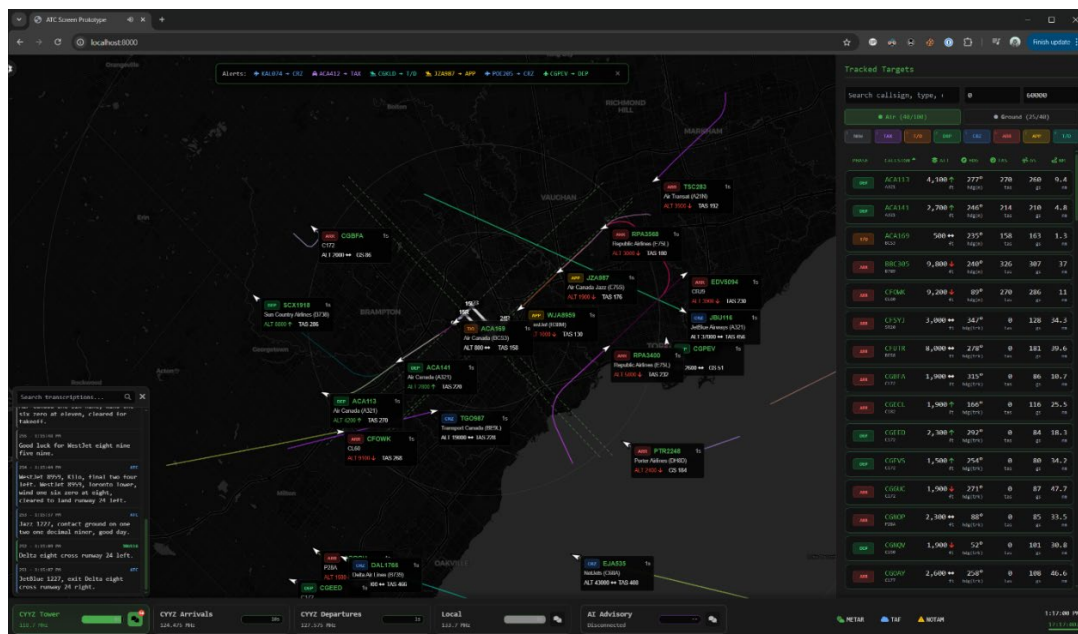


Ilustración 13 - Menú principal de Co-ATC

5.1 ADAPTACIÓN DE CO-ATC AL CONTEXTO DE LEBL

Co-ATC es un proyecto open-source que permite visualizar, gestionar y simular aeronaves mediante una interfaz web y una API REST. Co-ATC puede funcionar tanto con receptores ADS-B reales como con fuentes locales de datos JSON, lo que lo convierte en una herramienta ideal para entornos educativos o de desarrollo sin necesidad de hardware aeronáutico.

El programa se entrega con una configuración orientada a Toronto (CYYZ). Para que los cálculos de distancia, rumbo y filtros visuales sean coherentes con un escenario español, se modificó `co-atc-main/configs/config.toml`. El fragmento siguiente resume el cambio más relevante:

```

1 [adsb]
2 local_source_url = "http://127.0.0.1:9000/aircraft.json"
3
4 [station]
5 latitude = 41.2971
6 longitude = 2.0785
7 elevation_feet = 14
8 airport_code = "LEBL"

```

2. `local_source_url`: ahora apunta a `http://127.0.0.1:9000/aircraft.json`, un feed que servimos con `python3 -m http.server` para que Co-ATC tenga siempre un fichero disponible.`

4. `[station]`: define la latitud, longitud, elevación y el código ICAO de Barcelona. Con ello, las herramientas de Co-ATC calculan correctamente distancias a la estación, fases de vuelo y filtros geográficos en la interfaz.

Además, se añadió un fichero mínimo `co-atc-main/data/aircraft.json`:

```

1 {"now": 0, "messages": 0, "aircraft": []}

```

Este JSON vacío evita errores cuando el backend intenta leer un feed inexistente. Al servirlo mediante un HTTP local (ejemplo: `python3 -m http.server 9000` dentro de co-atc-main/data/`), garantizamos que el bucle de ingestión se mantenga activo y que las aeronaves inyectadas se integren en la vista web y en los históricos de SQLite.`

5.2 ORQUESTADOR DE TRÁFICO ADS-B

En la implementación final del proyecto, el concepto de *orquestador de tráfico simulado* planteado en la primera entrega es sustituido por un **orquestador de tráfico ADS-B real**, basado en la ingestión periódica de datos procedentes de una fuente externa de tráfico aéreo operativo.

En lugar de generar aeronaves de forma sintética mediante un script de simulación, el sistema emplea un **servicio auxiliar de ingesta ADS-B**, cuya función principal es consultar de forma periódica una API externa de tráfico aéreo y transformar la información recibida al formato esperado por el backend Co-ATC. Este servicio actúa como intermediario entre la fuente ADS-B real y el sistema de visualización y análisis.

5.3 IMPLEMENTACIÓN DEL FEED ADS-B (SERVICIO ADSB-FEED)

El feed se implementa en `services/adsb-feed/feed.py`. Este servicio consulta de forma periódica la API pública <https://opendata.adsb.fi/api/v3> y transforma los resultados al formato `aircraft.json` esperado por la API Co-ATC. El proceso es cíclico: en cada ciclo descarga el estado actual de todas

las aeronaves, normaliza los campos (altitud, velocidad, posición, señal, etc.) y escribe el fichero actualizado en /data/aircraft.json.

La normalización se realiza en el propio feed para asegurar consistencia entre lo que consume Co-ATC y lo que finalmente indexa Logstash. Además, el feed puede inyectar anomalías controladas (si se activan) mediante variables de entorno (ANOMALY_*). Esto permite simular ataques de spoofing, altitudes imposibles o velocidades extremas sin alterar la lógica del backend.

1) Configuración por entorno

El feed se parametriza con variables de entorno para hacerlo flexible:

```

1 # Coordenadas y radio de consulta.
2 lat = float(os.environ.get("LAT", "41.2971"))
3 lon = float(os.environ.get("LON", "2.0785"))
4 dist = float(os.environ.get("DIST", "80"))
5
6 # Intervalo en segundos entre peticiones.
7 interval = float(os.environ.get("INTERVAL", "2"))
8
9 # Ruta donde se escribe aircraft.json.
10 out_path = Path(os.environ.get("OUT", "/data/aircraft.json"))
11 out_path.parent.mkdir(parents=True, exist_ok=True)
12
13 # Log donde guardamos anomalías (JSONL).
14 anomaly_log_path = Path(os.environ.get("ANOMALY_LOG", "/data/adsb_feed_anomalies.log"))
15 anomaly_log_path.parent.mkdir(parents=True, exist_ok=True)
16
17 # Anomalías activas o no.
18 anomaly_enabled = os.environ.get("ANOMALY_ENABLED", "false").strip().lower() in {
19     "1",
20     "true",
21     "yes",
22     "y",
23     "on",
24 }
```

2) Normalización de datos ADS-B

El resultado es un dict limpio con claves estándar (hex, flight, alt_baro, gs, rssi, etc.).

3) Selección de anomalías

`pick_indices(count, max_count, rate, seed)`

Selecciona qué aeronaves serán modificadas ese ciclo.

Funciona así:

- Evalúa cada índice con una pseudo-probabilidad.
- Respeta ANOMALY_RATE y ANOMALY_MAX_PER_WRITE.
- Cambia cada 5 segundos para evitar patrones repetitivos.

Esto permite control fino de la “densidad” de anomalías.

`# pick_type(types, seed, hex_code)`

Elige el tipo de anomalía a aplicar de forma estable. Si no se definen tipos, usa un valor por defecto.

```

1 def map_aircraft(ac: Dict[str, Any]) -> Dict[str, Any]:
2     """Normaliza un avion del feed externo al formato esperado por co-atc."""
3     alt_baro = safe_alt(ac.get("alt_baro"))
4     alt_geom = safe_alt(ac.get("alt_geom", ac.get("alt_baro")))
5     gs = ac.get("gs") or ac.get("tas") or ac.get("ias")
6     heading = ac.get("true_heading") or ac.get("track") or ac.get("mag_heading")
7     baro_rate = ac.get("baro_rate") or ac.get("geom_rate")
8     return {
9         "hex": (ac.get("hex") or "").upper(),
10        "type": ac.get("type"),
11        "flight": (ac.get("flight") or "").strip(),
12        "r": ac.get("r"),
13        "t": ac.get("t"),
14        "desc": ac.get("desc"),
15        "alt_baro": alt_baro,
16        "alt_geom": alt_geom,
17        "gs": gs,
18        "tas": ac.get("tas"),
19        "ias": ac.get("ias"),
20        "mach": ac.get("mach"),
21        "track": ac.get("track"),
22        "true_heading": heading,
23        "mag_heading": ac.get("mag_heading"),
24        "baro_rate": baro_rate,
25        "geom_rate": ac.get("geom_rate"),
26        "squawk": ac.get("squawk"),
27        "emergency": ac.get("emergency"),
28        "category": ac.get("category"),
29        "lat": ac.get("lat"),
30        "lon": ac.get("lon"),
31        "nic": ac.get("nic"),
32        "rc": ac.get("rc"),
33        "seen_pos": ac.get("seen_pos"),
34        "version": ac.get("version", 2),
35        "nic_baro": ac.get("nic_baro"),
36        "nac_p": ac.get("nac_p"),
37        "nac_v": ac.get("nac_v"),
38        "sil": ac.get("sil"),
39        "sil_type": ac.get("sil_type"),
40        "gva": ac.get("gva"),
41        "sda": ac.get("sda"),
42        "alert": ac.get("alert"),
43        "spi": ac.get("spi"),
44        "mlat": ac.get("mlat", []),
45        "tisb": ac.get("tisb", []),
46        "messages": ac.get("messages"),
47        "seen": ac.get("seen"),
48        "rssi": ac.get("rssi"),
49    }

```

4) Inyección de anomalías

inject_anomaly(ac, anomaly_type, seed)

Aplica modificaciones según el tipo. Cada tipo está ajustado para cruzar umbrales SIEM reales:

- altitud_negativa → alt_baro < -100
- velocidad_excesiva → gs > 1200
- posible_spoofing → rssi > -10 y nic < 6, además nac_p, nac_v, sil, sda bajos

- squawk_emergencia → 7500/7600/7700
- salto_posicion → lat/lon desplazados dentro de límites válidos

Al final de cada anomalía, se fuerza también el patrón RSSI + distancia incoherente:

- RSSI fuerte (-5 a -1)
- Se desplaza lat/lon para simular distancia > 200 NM

Esto asegura que Logstash detecte alerta_rssi_distancia_incoherente.

5) Logging de anomalías

Cada anomalía se guarda en JSON Lines (uno por avión) en ANOMALY_LOG. Esto deja trazabilidad de qué campo fue alterado y cómo.

```

1 if hex_code:
2     last_anomaly_cycle[hex_code] = cycle_counter
3     modified_hexes.add(hex_code)
4     try:
5         with anomaly_log_path.open("a", encoding="utf-8") as fh:
6             detailed_changes = []
7             for field, values in changes.items():
8                 detailed_changes.append(
9                     {
10                        "field": field,
11                        "correct_value": values[0],
12                        "modified_value": values[1],
13                    })
14             record = {
15                 "ts": ts,
16                 "id_icao": item.get("hex"),
17                 "anomaly_type": anomaly_type,
18                 "changes": detailed_changes,
19             }
20             fh.write(json.dumps(record) + "\n")
21     except Exception:
22         # Si falla el log, seguimos igual.
23         pass

```

6) Bucle principal

El ciclo principal hace:

1. GET a <https://opendata.adsb.fi/api/v3/lat/<lat>/lon/<lon>/dist/<dist>>
2. Mapeo de aeronaves con map_aircraft
3. Inyección de anomalías (si está activado)
4. Escritura de aircraft.json
5. Sleep por INTERVAL

El resultado es un aircraft.json vivo y actualizado que Co-ATC consume como fuente ADS-B local. En entorno Docker, la URL usada es <http://adsb-feed:9000/aircraft.json> (red interna de contenedores).

5.4 ESCENARIO OPERATIVO Y CONTROL DEL RITMO DE INGESTA

El laboratorio opera sobre un **escenario basado en tráfico ADS-B real**, eliminando la generación de vuelos simulados con rutas y horarios predefinidos. Co-ATC refleja en tiempo casi real el estado de las aeronaves visibles en la zona de interés, proporcionando un entorno más realista para el análisis SIEM.

Para garantizar la estabilidad del sistema, el servicio de adquisición ADS-B incorpora mecanismos de **control del ritmo de ingesta**, ajustando la frecuencia de consulta al feed externo y la actualización del estado de las aeronaves. Este control evita sobrecargas y mantiene una latencia reducida entre la recepción del dato y su disponibilidad en el pipeline Elastic.

Adicionalmente, el procesamiento en Logstash aplica **deduplicación de estados consecutivos**, reduciendo eventos redundantes sin perder información relevante. De forma opcional, el sistema permite la **inyección controlada de anomalías** para la validación de reglas SIEM, manteniendo siempre un comportamiento estable del escenario operativo.

5.5 FLUJO COMPLETO Y RELACIÓN CON ELASTIC STACK

En funcionamiento normal, Co-ATC mantiene el estado actualizado de las aeronaves a partir del flujo ADS-B real y lo expone mediante el endpoint `/api/v1/aircraft`. Este endpoint constituye la fuente de datos para el pipeline SIEM del laboratorio.

Logstash consulta periódicamente dicha API y procesa la información recibida, separando los datos por aeronave y aplicando tareas de normalización, deduplicación y detección de anomalías. Los eventos resultantes se indexan en Elasticsearch en índices diarios, permitiendo su almacenamiento y análisis histórico.

Kibana utiliza estos índices para la creación de dashboards, visualizaciones y consultas orientadas tanto a la monitorización del tráfico aéreo como a la identificación de comportamientos anómalos. De forma complementaria, los eventos etiquetados como anomalías se registran en logs estructurados, que son consumidos directamente por la interfaz de Co-ATC para mostrar alertas operativas en tiempo casi real.

Este flujo integra de forma coherente Co-ATC y Elastic Stack, proporcionando un entorno SIEM funcional y alineado con los objetivos del laboratorio.

6. EJEMPLO PRÁCTICO DE FUNCIONAMIENTO Y OPERACIÓN

Este apartado describe un caso de uso real dentro del laboratorio para mostrar cómo se detecta una anomalía, cómo se visualiza en Kibana y cómo se notifica al controlador en Co-ATC. El objetivo es conectar la teoría con la operación diaria del sistema.

6.1 ESCENARIO DE EJEMPLO

Para mostrar el funcionamiento real del laboratorio, se describe un escenario operativo completo en el que una anomalía es inyectada, detectada y visualizada por el sistema SIEM. El objetivo de este ejemplo es demostrar la coherencia del flujo entre el feed ADS-B, el backend de Co-ATC, Logstash, Elasticsearch y Kibana, y cómo esa detección llega finalmente al controlador aéreo (ATC).

Para iniciar el laboratorio desde la raíz del repositorio, lo primero es levantar todos los servicios con Docker Compose. El comando recomendado es: **#docker-compose up -d --build**

```

♦ mario on 12:48:54 ♦master
♦ f ♦ ~/./././ADS-B_SIEM } ♦ sudo docker-compose up -d
[sudo] password for mario:
Creating network "ads-b_siem_default" with the default driver
Creating ads-b_siem_elasticsearch_1 ... done
Creating ads-b_siem_adsb-feed_1 ... done
Creating ads-b_siem_co-atc_1 ... done
Creating ads-b_siem_logstash_1 ... done
Creating ads-b_siem_kibana_1 ... done

♦ mario on 12:49:24 ♦master
♦ f ♦ ~/./././ADS-B_SIEM } sudo docker-compose ps

```

Name	Command	State	Ports
ads-b_siem_adsb-feed_1	/app/start.sh	Up	0.0.0.0:9000->9000/tcp,:::9000->9000/tcp
ads-b_siem_co-atc_1	/app/co-atc -config /app/c ...	Up	0.0.0.0:8000->8000/tcp,:::8000->8000/tcp
ads-b_siem_elasticsearch_1	/bin/tini -- /usr/local/bi ...	Up (healthy)	0.0.0.0:9200->9200/tcp,:::9200->9200/tcp, 9300/tcp
ads-b_siem_kibana_1	/bin/tini -- /usr/local/bi ...	Up	0.0.0.0:5601->5601/tcp,:::5601->5601/tcp
ads-b_siem_logstash_1	/usr/local/bin/docker-entr ...	Up (healthy)	0.0.0.0:5044->5044/tcp,:::5044->5044/tcp, 0.0.0.0:9600->9600/tcp,:::9600->9600/tcp

Ilustración 14 - Despliegue y verificación de servicios mediante Docker Compose

Una vez en marcha, el servicio adsb-feed comienza a descargar datos reales desde opendata.adsb.fi y genera aircraft.json.

```

adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:15] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:17] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | [INFO] adsb-feed: 49 aeronaves
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:19] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | [INFO] adsb-feed: 48 aeronaves
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:21] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:23] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | [INFO] adsb-feed: 48 aeronaves
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:25] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | [INFO] adsb-feed: 48 aeronaves
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:27] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:29] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | [INFO] adsb-feed: 48 aeronaves
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:31] "GET /aircraft.json HTTP/1.1" 200 -
adsb-feed_1 | 172.18.0.4 - - [30/Dec/2025 14:59:33] "GET /aircraft.json HTTP/1.1" 200 -

```

Ilustración 15 - Logs del servicio adsb-feed mostrando la obtención periódica de datos y el conteo de aeronaves activas

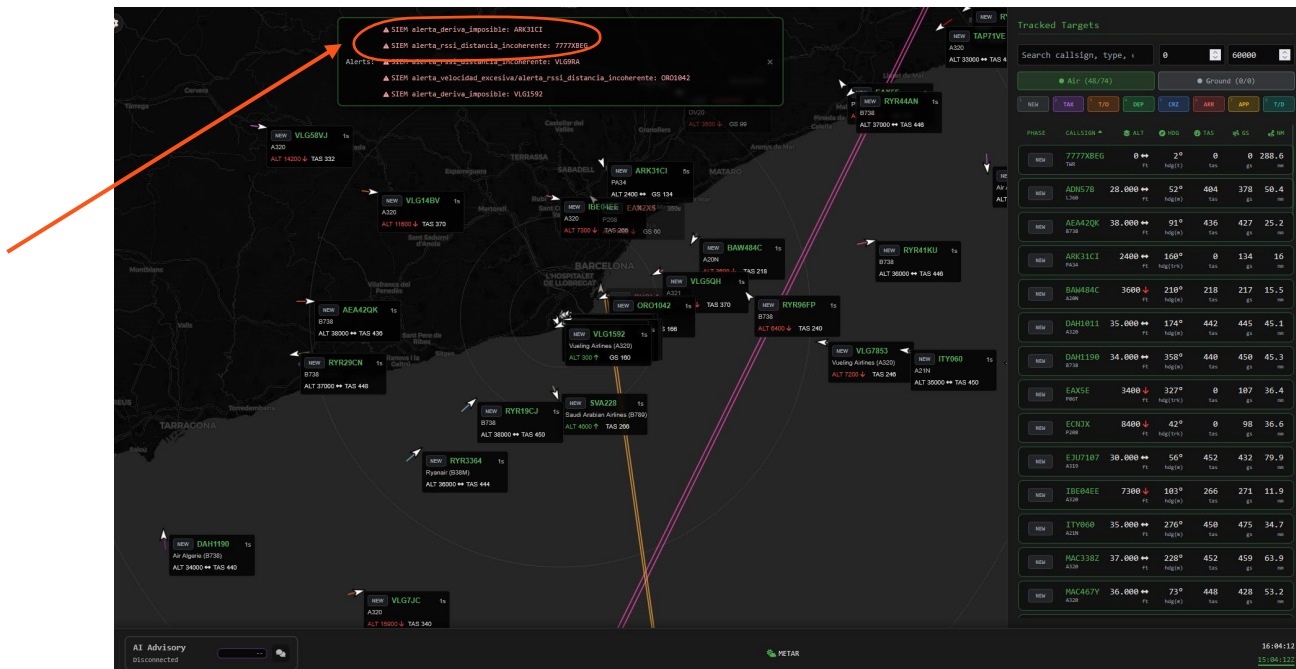
En este escenario se observará la inyección de anomalías en el servicio adsb-feed del tipo: el valor de velocidad de tierra se eleva por encima de 1200 nudos y, además, se fuerza un patrón de señal fuerte con distancia incoherente.

Co-ATC consume ese feed y expone el estado en su API REST. Para comprobarlo, se puede consultar: **`#curl http://localhost:8000/api/v1/aircraft | head`**

```
% mario on 16:02:02 @master
% { % ~/.../ADS-B_SITEM | curl http://localhost:8000/api/v1/aircraft | head
% Total % Received % Xferd Average Speed Time Time Time Current
                               Dload Upload Total Spent Left Speed
0      0      0      0      0      0 --:--:-- --:--:-- --:--:-- 0["timestamp": "2025-12-30T15:02:06.956603774Z", "count": 441, "counts":
441], "aircraft": [{"hex": "440170", "flight": "EJU91RA", "airline": "", "status": "active", "last_seen": "2025-12-30T15:02:05Z", "on_ground": false, "date_la
", "distance": 1, "adsb": {"hex": "440170", "type": "adsb_icao", "flight": "EJU91RA", "r": "OE-LQM", "t": "A319", "alt_baro": -100, "alt_geom": 200, "gs": 131.6,
": 245.77, "track_rate": 0, "roll": 0, "mag_heading": 250.84, "true_heading": 252.96, "baro_rate": -256, "geom_rate": -672, "squawk": "1000", "category": "A3", "
": 0, "lat": 41.304357, "lon": 2.099637, "nic": 8, "rc": 186, "seen_pos": 0.52, "r_dst": 0, "r_dir": 0, "version": 2, "nic_baro": 1, "nac_p": 9, "nac_v": 1, "sil": 3, "si
b": [], "messages": 194579759, "seen": 0, "rsst": -17.7, "source_type": "local", "is_simulated": false}, {"hex": "34469U", "flight": "VLG41PY", "airline": "", "
d": false, "date_landed": null, "date_takeoff": null, "created_at": "2025-12-30T12:35:31Z", "distance": 79.8, "adsb": {"hex": "34469U", "type": "adsb_icao",
om": 37950, "gs": 451.8, "ias": 244, "tas": 436, "mach": 0.772, "wd": 0, "ws": 0, "oat": 0, "tat": 0, "track": 353.65, "track_rate": 0, "roll": 0, "mag_heading": 354.9,
00", "category": "A3", "nav_qnh": 0, "nav_altitude_mcp": 0, "nav_altitude_fms": 0, "nav_heading": 0, "lat": 42.500519, "lon": 1.320337, "nic": 8, "rc": 186, "seen
c_v": 1, "sil": 3, "sil_type": "perhour", "gva": 2, "sda": 2, "alert": 0, "spi": 0, "mlat": [], "tsib": [], "messages": 160548788, "seen": 0, "rs": -15, "source_type
", "airline": "", "status": "signal_lost", "last_seen": "2025-12-30T14:43:26Z", "on_ground": false, "date_landed": null, "date_takeoff": null, "created_at":
": "adsb_icao", "flight": "SWR4TD", "r": "HB-TON", "t": "A321", "alt_baro": 270400, "alt_geom": 27525, "gs": 444.7, "ias": 317, "tas": 466, "mach": 0.788, "wd": 0, "
ading": 59.24, "true_heading": 61.72, "baro_rate": 1472, "geom_rate": 1440, "squawk": "1000", "category": "A3", "nav_qnh": 0, "nav_altitude_mcp": 0, "nav_altit
rc": 186, "seen_pos": 0.067, "r_dst": 0, "r_dir": 0, "version": 2, "nic_baro": 1, "nac_p": 9, "nac_v": 1, "sil": 3, "sil_type": "perhour", "gva": 2, "sda": 2, "alert":
ic": -14, "source_type": "local", "is_simulated": false}, {"hex": "4D239S", "flight": "RAM938", "airline": "Royal Air Maroc", "status": "signal_lost", "last
date_takeoff": null, "created_at": "2025-12-30T13:28:24Z", "distance": 79.8, "adsb": {"hex": "4D239S", "type": "adsb_icao", "flight": "RAM938", "r": "9H-AMM
7, "tas": 442, "mach": 0.78, "wd": 0, "ws": 0, "oat": 0, "tat": 0, "track": 76.9, "track_rate": 0, "roll": 0, "mag_heading": 77.17, "true_heading": 79.65, "baro_rate"
```

Ilustración 16 - Despliegue y verificación de servicios mediante Docker Compose

En este punto es recomendable abrir el navegador en <http://localhost:8000> y hacer una captura del radar de Co-ATC (vista principal). Si se tiene anomalías activadas, más adelante aparecerán alertas en la barra superior.



*Ilustración 17 - Alertas que indica Co-ATC (rodeadas): SIEM alerta_deriva_imposible: **ARK31CI** && SIEM alerta_rssi_distancia_incoherente: **7777XBEG***

Desde el punto de vista del usuario, Co-ATC muestra un tráfico aparentemente normal, pero con aeronaves que presentan parámetros físicamente improbables o imposibles. Esta situación emula

un ataque o una interferencia sobre los datos ADS-B y permite evaluar si el sistema lo detecta correctamente.

El proceso completo ocurre en tiempo casi real. El feed genera el aircraft.json con el avión modificado y Co-ATC lo incorpora como parte del estado general de la aeronave.

A continuación, Logstash consulta el endpoint /api/v1/aircraft, separa los datos por avión y aplica las reglas de detección. En este caso, se generan dos etiquetas: alerta_velocidad_excesiva y alerta_rssi_distancia_incoherente. El evento resultante se indexa en Elasticsearch bajo el índice diario adsb-siem-YYYY.MM.dd, quedando inmediatamente disponible para consulta y visualización.

Por ello, el siguiente paso es comprobar que Logstash está recogiendo datos desde Co-ATC y enviándolos a Elasticsearch. Para ello, se revisa su log: **#docker-compose logs -f logstash**

```

♦ mario on 16:58:57 ~master
♦ { * - ./ADS-B_SIEM } sudo docker-compose logs -f logstash
Attaching to adsb-siem_logstash_1
logstash_1 | Using bundled JDK: /usr/share/logstash/jdk
logstash_1 | Sending Logstash logs to /usr/share/logstash/logs which is now configured via log4j2.properties
logstash_1 | [2025-12-30T11:49:34.836] [WARN] [deprecation.logstash.runner] NOTICE: Running Logstash as supervisor is not recommended and won't be allowed in the future. Set 'allow_superuser' to 'false' to avoid startup errors in future releases.
logstash_1 | [2025-12-30T11:49:34.841] [INFO] [logstash.runner] | Log4j configuration path used is: /usr/share/logstash/config/log4j2.properties
logstash_1 | [2025-12-30T11:49:34.843] [INFO] [logstash.runner] | Starting Logstash {"logstash.version":"8.14.1", "ruby.version":"ruby 9.4.7.0 (3.1.4) 2024-04-29 597ff08ac1 OpenJDK 64-Bit Serv
or VM 17.0.11+9 on 17.0.11+9 -indy #18 (x86_64-linux)"}
logstash_1 | [2025-12-30T11:49:34.844] [INFO] [logstash.runner] | JVM bootstrap flags: [-Xms1g, -Xmx1g, -Djava.awt.headless=true, -Dfile.encoding=UTF-8, -Djruby.compile.invokedynamic=true, -XX:+He
adDumpOnOutOfMemoryError, -Djava.security.egd=file:/dev/urandom, -Dlog4j2.isThreadContextMapInheritable=true, -Dlogstash.jackson.stream-read-constraints.max-string-length=2080000000, -Dlogstash.jackson.stream-r
ead-constraints.max-number-length=10000, -Dls.cgroup.cpuacct.path.override=/, -Dls.cgroup.cpu.path.override=/, -Xms512m, -Xmx512m, -Djruby.regex.interruptible=true, -Djdk.io.file.enableAOS=true, --add-exports
jdk.compiler/com.sun.tools.javac.api=ALL-UNNAMED, --add-exports=jdk.compiler/com.sun.tools.javac.util=ALL-UNNAMED, --add-opens=java.base/java.io=ALL-UNNAMED, --add-opens=java.base/java.nio.channels=ALL-UNNAMED, --add-opens=java.base/sun.nio.ch=ALL-UNNAMED, --add-opens=java.management/sun.management=ALL-UNNAMED, -Dio.netty allocator.maxOrder=11]
logstash_1 | [2025-12-30T11:49:34.846] [INFO] [logstash.runner] | Jackson default value override 'logstash.jackson.stream-read-constraints.max-string-length' configured to '2080000000'
logstash_1 | [2025-12-30T11:49:34.846] [INFO] [logstash.runner] | Jackson default value override 'logstash.jackson.stream-read-constraints.max-number-length' configured to '10000'
logstash_1 | [2025-12-30T11:49:34.850] [INFO] [logstash.settings] | Creating directory {settings=>"path.queue", .path=>"/usr/share/logstash/data/queue"}
logstash_1 | [2025-12-30T11:49:34.854] [INFO] [logstash.settings] | Creating directory {settings=>"path.dead_letter_queue", .path=>"/usr/share/logstash/data/dead_letter_queue"}
logstash_1 | [2025-12-30T11:49:35.018] [INFO] [logstash.agent] | No persistent UUID file found. Generating new UUID {uuid=>"5ddcb1c4-e280-40c0-8a2a-233f61a5f4f8", .path=>"/usr/share/logstash/dat
a/uuid"}

```

6.2 DETECCIÓN EN KIBANA (DISCOVER)

Desde Kibana, el operador puede verificar la anomalía usando Discover.

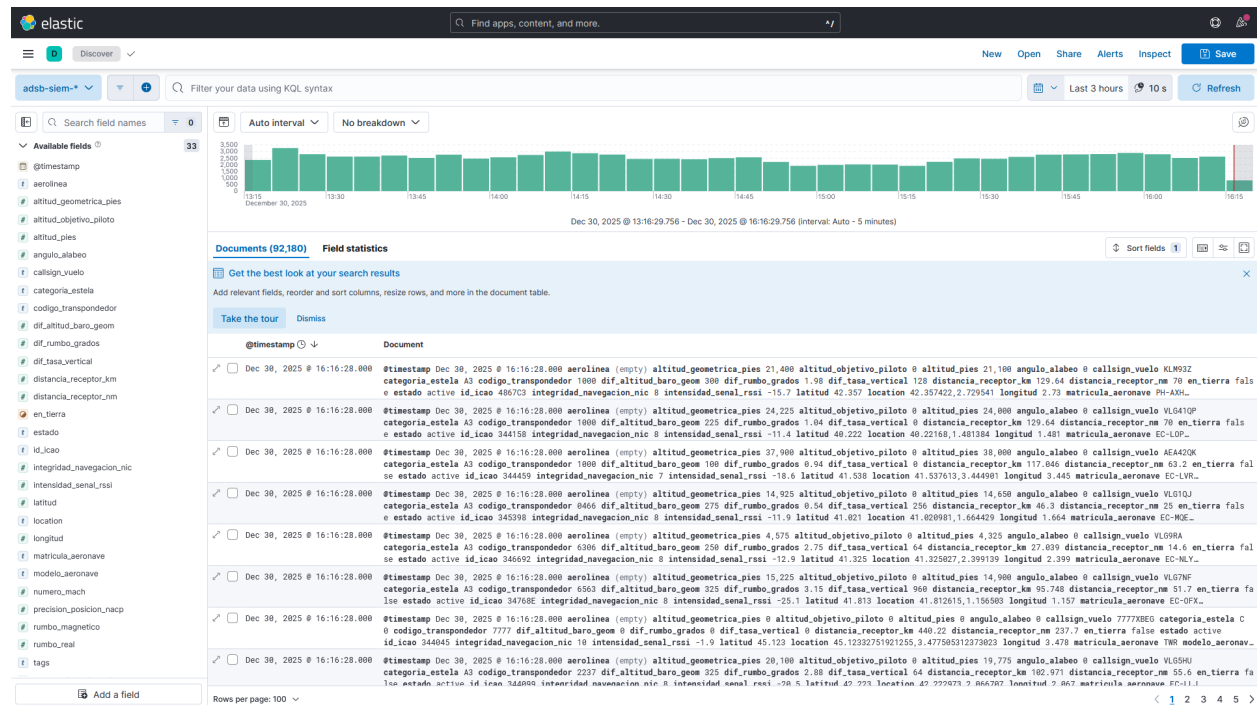


Ilustración 17- Interfaz Discover de Kibana para la inspección y filtrado de eventos ADS-B normalizados.

Al seleccionar el data view adsb-siem-* y aplicar un filtro doble KQL como tags: "alerta_velocidad_excesiva", se muestran los eventos anómalos generados por estas dos anomalías en los últimos minutos.

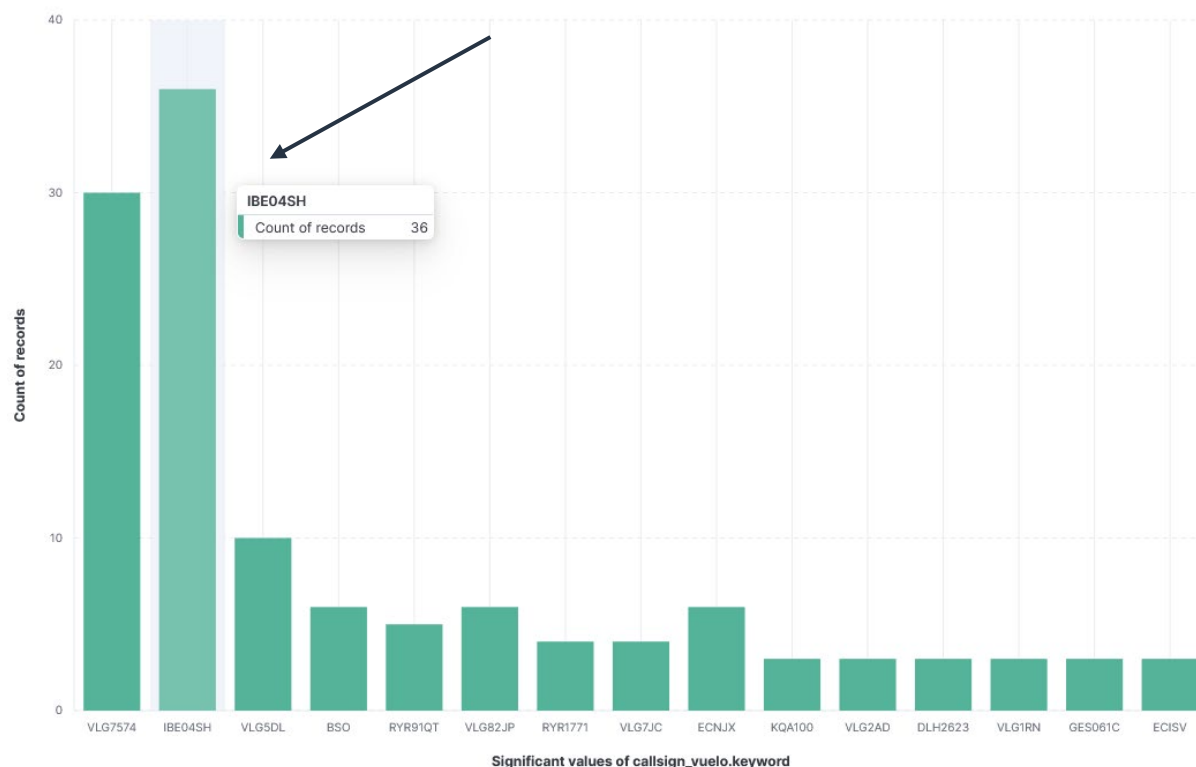


Ilustración 18 - Gráfico de barras que muestra el número de eventos por callsign (callsign_vuelo.keyword) dentro del rango temporal seleccionado en Kibana.

Cada barra representa un avión/callsign y su altura indica cuántos registros se han generado para ese vuelo. En el ejemplo, IBE04SH es el callsign con más eventos (36), seguido por VLG7574 (~30), mientras que el resto tiene presencia mucho menor.

Volviendo a las alertas de Co-ATC, para verificar que el stack funciona correctamente se comprueba mediante filtros KQL de anomalías (etiquetadas por logstash) e introduciendo los callsigns de las aeronaves que indica el frontend Co-ATC:

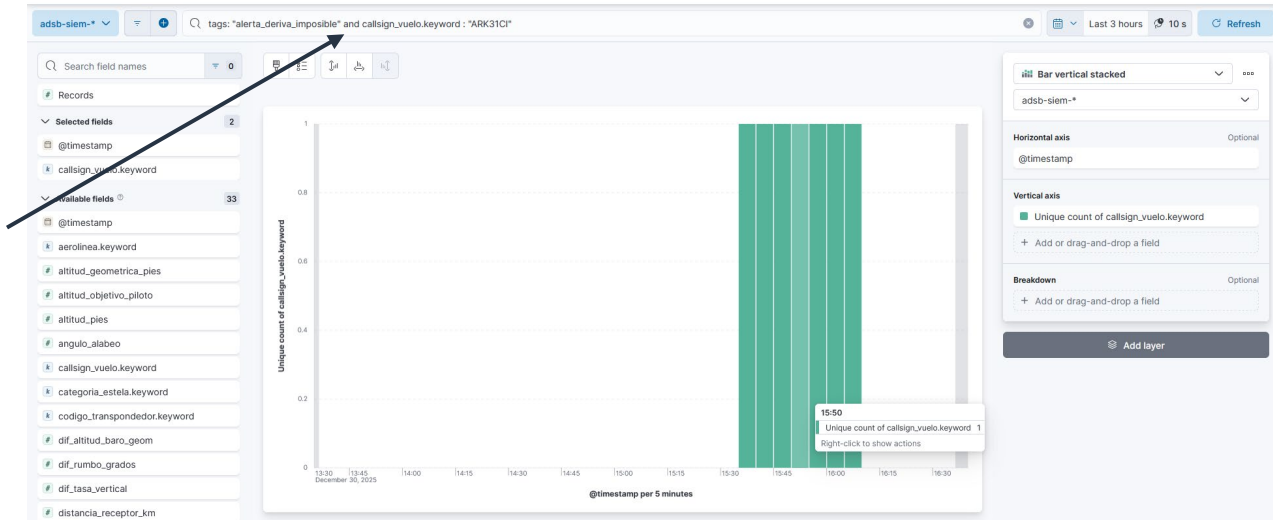


Ilustración 19 - Gráfico de barras en Kibana para la monitorización temporal de alertas específicas por aeronave.

Esta vista permite validar rápidamente la coherencia del evento y confirmar que los datos que el SIEM analiza coinciden con lo que Co-ATC está mostrando.

La evidencia confirma que el stack funciona de extremo a extremo. Kibana muestra eventos agregados en tiempo real sobre el índice adbsb-siem-*, demostrando que la ingesta y el procesamiento en Logstash son correctos. A la vez, Co-ATC visualiza las alertas SIEM generadas, lo que verifica que las reglas se aplican y que el sistema notifica al operador. En conjunto, se valida el flujo completo desde el feed ADS-B hasta la detección y visualización operativa de anomalías.

7. RELACIÓN CON LA ASIGNATURA Y RESUMEN DE LO REALIZADO

En esta práctica se ha construido un laboratorio completo que implementa un SIEM funcional sobre **tráfico ADS-B real**. El sistema se despliega con Docker Compose e integra cinco servicios principales: adbsb-feed, Co-ATC, Logstash, Elasticsearch y Kibana.

Desde el punto de vista de la asignatura, el trabajo conecta con varios bloques. En administración de servicios, el laboratorio exige arrancar y detener servicios, revisar su estado y gestionar logs por contenedor, lo que reproduce tareas reales en algunas pls. En monitorización y gestión de logs, Logstash, Elasticsearch y Kibana representan una canalización completa: ingesta estructurada, transformación, deduplicación, indexación y visualización, equivalente a un sistema de logging avanzado. En seguridad, las reglas SIEM implementadas (altitud negativa, velocidad excesiva, spoofing por RSSI/NIC, squawk de emergencia, etc.) justifican la detección basada en correlación y anomalías, alineada con los contenidos de auditoría y respuesta ante incidentes. En redes, el proyecto utiliza APIs HTTP internas, expone puertos y maneja un flujo de datos distribuido entre contenedores. Y en virtualización/contenerización, Docker permite aislar servicios, asegurar reproducibilidad y simplificar el despliegue, que es una competencia directa del temario.

En términos de resultados, se demuestra que el stack funciona de extremo a extremo: el tráfico real se refleja en Co-ATC, las anomalías se detectan en Logstash, quedan indexadas en Elasticsearch, se visualizan en Kibana y aparecen como alertas operativas en la interfaz. Esto valida tanto la implementación técnica como la conexión con los objetivos formativos: administración de sistemas, gestión de servicios, seguridad, monitorización y operación de infraestructuras.

8. CONCLUSIONES Y PASOS SIGUIENTES

La realización de este proyecto ha permitido validar con éxito la hipótesis inicial de que Elastic Stack constituye una solución viable, flexible y potente para actuar como un sistema de Gestión de Eventos e Información de Seguridad (SIEM) en entornos críticos como la aviación, aportando una capa de seguridad necesaria a protocolos vulnerables por diseño como el ADS-B. A diferencia de la propuesta teórica inicial, esta entrega final ha logrado crear un laboratorio funcional y reproducible mediante contenedores Docker, capaz de transformar una ingesta de datos de tráfico aéreo real en información operativa de seguridad.

Uno de los pilares fundamentales del éxito de esta implementación ha sido el diseño de una arquitectura modular donde Logstash no se limita a la mera recolección de logs, sino que actúa como un motor de procesamiento lógico avanzado. Al trasladar la lógica de detección de amenazas a la fase de ingesta, el sistema es capaz de evaluar en tiempo casi real la consistencia física de los datos, identificando anomalías críticas como velocidades imposibles, altitudes negativas o incoherencias entre la intensidad de la señal (RSSI) y la distancia, las cuales suelen ser indicativos de ataques de *spoofing* o errores de integridad. La capacidad del servicio de adquisición desarrollado para inyectar anomalías controladas sobre tráfico legítimo ha servido para verificar empíricamente la eficacia de estas reglas, demostrando que el sistema puede distinguir eventos maliciosos dentro de un entorno de operaciones normales sin generar falsos positivos excesivos.

Asimismo, el proyecto ha logrado cerrar el ciclo de inteligencia de seguridad mediante la integración efectiva del SIEM con la interfaz operativa del controlador en Co-ATC, simulando este el software corporativo de cualquier sistema de control aéreo real. Esta característica trasciende la administración de sistemas tradicional, ya que no solo almacena y visualiza datos en los dashboards de Kibana para el análisis forense, sino que notifica las alertas directamente en el radar del operador en el momento en que ocurren. Este enfoque demuestra cómo las herramientas *Open Source* de monitorización pueden converger con los sistemas de operación en tiempo real, mejorando la conciencia situacional y la capacidad de respuesta ante incidentes sin obligar al usuario a cambiar de contexto visual.

Desde una perspectiva académica, el desarrollo de este laboratorio ha supuesto la aplicación práctica e integrada de las competencias clave de la asignatura, abarcando desde la virtualización y orquestación de servicios con Docker Compose hasta la gestión avanzada de logs estructurados

y la seguridad de la información. La infraestructura resultante cumple con los requisitos no funcionales de reproducibilidad y modularidad, permitiendo aislar cada componente del stack para facilitar su mantenimiento y escalabilidad. En definitiva, este trabajo demuestra que es posible mitigar las vulnerabilidades inherentes a las comunicaciones ADS-B no cifradas mediante una arquitectura de monitorización inteligente y centralizada, consolidando a Elastic Stack como una herramienta idónea para la detección de intrusiones y la gestión de incidentes en infraestructuras críticas simuladas.

9. REFERENCIAS Y BIBLIOGRAFÍA

- ✚ Elastic. (2023, 17 de abril). Flightwatching helps carriers reduce CO₂ emissions and save \$100,000+ per aircraft per year, with Elastic. <https://www.elastic.co/blog/flightwatching-helps-carriers-with-elastic>
- ✚ La Roche, A., & Beeching, B. (2025, 18 de marzo). Elasticsearch in the aviation industry: A game-changer for data management. Elastic. <https://www.elastic.co/blog/elasticsearch-data-management-aviation>
- ✚ Segurilatam. (2017, 18 de mayo). Ciberseguridad en aeropuertos y compañías de aviación civil. https://www.segurilatam.com/seguridad-por-sectores/puertos-y-aeropuertos/ciberseguridad-en-aeropuertos-y-companias-de-aviacion-civil_20170518.html
- ✚ Controladores Aéreos. (2024, 8 de enero). ADS-B: La Revolución en la Vigilancia del Tráfico Aéreo en España. <https://controladoresaereos.es/ads-b-la-revolucion-en-la-vigilancia-del-traffic-aereo-en-espana/>
- ✚ Murisa, W. (2025). A Conceptual SOC Framework for Air Traffic Management Systems. *Proceedings of ICISSP 2025*. Scitepress. PDF: <https://www.scitepress.org/Papers/2025/131742/131742.pdf>
- ✚ Kosho. (s. f.). Flight Track — ADS-B Signal Visualization with Elastic Stack [Repositorio GitHub]. <https://github.com/kosho/flight-track>
- ✚ Gilbert, G. (2017, 27 de febrero). FAA Cautions about Transponder and ADS-B Testing. *AINonline*. <https://www.ainonline.com/aviation-news/aerospace/2017-02-27/faa-cautions-about-transponder-and-ads-b-testing>
- ✚ W. Ahmed, N.A. Bhatti, A. Masood, A.A. Alharbi & A. Alotaibi. (2024). *Advancements in ADS-B Security: A Comprehensive Survey of Vulnerabilities, Mitigation Strategies, System Requirements and Emerging Research Trends*. Preprints.org. <https://doi.org/10.20944/preprints202405.0586.v1>
- ✚ J. Manzoor. (2024). *Evaluating security and performance of open-source SIEM systems*. PMC. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10977669/>
- ✚ M. Sheeraz. (2024). *Revolutionizing SIEM Security: An Innovative Correlation Engine for Multi-Layered Attack Detection*. *Sensors*, 24(15), 4901. MDPI. <https://doi.org/10.3390/s24154901>

- ✚ Y.W. Law & J. Slay. (2022). *SIEM4GS: Security Information and Event Management for a Virtual Ground Station Testbed*. University of South Australia / SmartSat CRC. <https://ywlaw.github.io/pub/law2022siem4gs.pdf>
 - ✚ Aymard Cuello, M. (2019). *Security Monitoring System Applied to IoT*. Tesis Máster, Universidad Politécnica de Madrid. https://oa.upm.es/65634/1/TESIS_MASTER_MARIANNE_AYMARD_CUELLO.pdf
 - ✚ X. Ying, J. Mazer, G. Bernieri, M. Conti, L. Bushnell & R. Poovendran. (2019). *Detecting ADS-B Spoofing Attacks using Deep Neural Networks*. arXiv. <https://arxiv.org/abs/1904.09969>
 - ✚ M. Ngamboé, J.-S. Marrocco, J.-Y. Ouattara, J.M. Fernandez & G. Nicolescu. (2023). *CABBA: Compatible Authenticated Bandwidth-efficient Broadcast protocol for ADS-B*. arXiv. <https://arxiv.org/abs/2312.09870>
 - ✚ S. Akerman, E. Habler & A. Shabtai. (2019). *VizADS-B: Analyzing Sequences of ADS-B Images Using Explainable Convolutional LSTM Encoder-Decoder to Detect Cyber Attacks*. arXiv. <https://arxiv.org/abs/1906.07921>
 - ✚ “AI-driven SIEM that is open source and affordable”. (s. f.). Elastic. <https://www.elastic.co/security/siem>
-